

DevOps & Delivery Pipeline

CI/CD, containerization, and release engineering for **Meridian** — a FastAPI + React studio-management platform shipped as a single versioned container image. Every change is linted, tested against an ephemeral database, type-checked, built, and (on `main`) published to the container registry — gated locally by a pre-commit suite and a CI-mirroring script so a green local run predicts a green pipeline.

GitHub Actions	Docker / Buildx	Compose	GHCR
Nginx (TLS)	PostgreSQL 15	Alembic	pre-commit
Ruff / ESLint	pytest / Jest	BuildKit cache	SonarQube

Environment	Artifact	Version
Northstar Animation Studios	One versioned image, two services	Meridian 1.5.1 (single-source)

Summary. Meridian is delivered as a single container image that serves two roles — a FastAPI backend under Uvicorn and a React frontend served by Nginx. A GitHub Actions pipeline validates every change against an ephemeral PostgreSQL database, and only commits to `main` build and publish a versioned image to the registry. Deployment to the production host is a deliberate, operator-initiated pull. Quality is enforced left-to-right: a pre-commit suite catches issues on every commit, a local script mirrors CI exactly, and the server-side pipeline is the final gate. This document covers the DevOps surface only; application features are documented separately.

1 — Pipeline at a Glance

The pipeline has three stages: continuous integration on every change, image publishing on main only, and an operator-initiated deploy. This is continuous delivery *to the registry* with deliberate promotion *to the box*.

Stage 1 — Continuous integration

Runs on every push or PR to main or develop. A health-gated postgres:15-alpine service container backs the backend tests; Python and Node dependency caches are restored; then the checks run in sequence. Any failure — including a single ESLint warning — fails the run.

```
ruff check # Python lint + security rules
pytest + coverage # against ephemeral postgres:15-alpine
ESLint --max-warnings=0 # zero-warning policy
tsc typecheck # TypeScript types
Jest # frontend unit tests
production build # React build must succeed
```

Stage 2 — Image publish (main only)

When the checks pass on main, the version is read from version.json, the production image is built with Buildx using the GitHub Actions layer cache, tagged, and pushed to the registry behind a retry/timeout wrapper to ride out registry flakiness.

1	Resolve version	Read version.json to determine the image tag.
2	Build	docker buildx with Dockerfile.prod; cache-from/to = GitHub Actions (mode=max).
3	Tag	prod-latest and prod-X.Y.Z.
4	Push	To the container registry; 3 retries, 300s timeout.

Stage 3 — Deploy

Deployment is operator-initiated on the production host: a single make target pulls the new image and recreates the services. Nothing is built on the production box.

```
# on the production host
make prod-pull # docker compose pull && docker compose up -d
```

2 — Branching & Environments

Direct commits to `main` are blocked by a pre-commit branch guard — all changes arrive by pull request. The two long-lived branches are treated differently: `develop` and PRs get validation only, while `main` additionally builds and publishes the image.

→	feature/*	Opened as a PR into <code>develop</code> .
→	develop	CI validates (lint, test, typecheck, build); no publish.
→	main	CI validates, then builds and pushes the versioned image.
→	registry	Production host pulls the published image on demand.

Environment comparison

	Development	Production
Compose file	<code>docker-compose.dev.yml</code>	<code>docker-compose.prod.yml</code>
Backend	built locally, <code>--reload</code> , <code>bind-mounts</code>	registry image <code>prod-latest</code> , <code>loopback-bound</code>
Frontend	CRA dev server + nginx TLS sidecar	same registry image running nginx
DB seeding	<code>AUTO_INIT_DB / SEED_* = true</code>	<code>all = false</code>
Certificates	self-signed (make certs)	mounted CA-signed certs
Migrations	Alembic upgrade head on boot	Alembic upgrade head on boot

There is no dedicated staging tier today — promotion is `dev` → `CI` → `prod`. A staging overlay that runs the built image against disposable data before a production pull is the natural next step.

3 — Production Runtime Topology

A single image serves two roles. The backend service runs Uvicorn; the frontend service runs Nginx from the same image (the built React app lives in the image's web root). The benefits: one `docker compose pull` updates both services at once, and there is nothing to build on the production host.

Topology

Component	Detail
Clients	Browser + PyQt6 desktop client (meridian-desktop)
frontend service	Nginx, :3443 TLS (:3000 redirects 301); HSTS, CSP, rate-limit 10r/s; SPA fallback
backend service	FastAPI / Uvicorn, bound to 127.0.0.1:8000 only; Alembic on boot; scheduler, SSE, websockets
database	postgres:15-alpine, named volume, health-gated
integrations	Cascade pipeline, LDAP / Active Directory, SMTP, studio storage array (TrueNAS)

The backend port is bound to **loopback only**, so all external traffic is forced through Nginx's TLS, CSP, and rate limiting — the backend is never directly reachable from outside the host. An optional Compose overlay adds Kerberos / Active Directory single sign-on (keytab + `krb5.conf`); rollback is simply not applying the overlay.

4 — Image Build Pipeline

The production image is a multi-stage, BuildKit-cached build. Toolchains stay in early stages so the final runtime image is lean and runs as a non-root user.

Multi-stage build

Stage	Base	Does
1 — frontend-build	node:20-alpine	npm ci + production build; cache-mounted npm dir
2 — python-deps	python:3.11-slim	build toolchain + libldap/sasl/krb5; pip install (cache mount)
3 — runtime	python:3.11-slim	+ nginx + ffmpeg; non-root appuser (uid 1001); Alembic baked in

The runtime stage copies the built frontend from stage 1 and the installed Python packages from stage 2, then runs as either Uvicorn (backend) or Nginx (frontend) depending on the Compose service.

Why it is fast and safe

- **Cache mounts.** `RUN --mount=type=cache` persists pip wheels and the npm cache across rebuilds without bloating layers; in CI these export to the GitHub Actions cache (`mode=max`).
- **Cache-friendly ordering.** Lockfiles are copied and dependencies installed *before* source, so a source-only change skips the expensive install layers.
- **Lean runtime.** Build toolchains never reach the final image, which ships only runtime libraries (libkrb5, ffmpeg, nginx).
- **Non-root.** The container runs as appuser (uid 1001), not root.
- **Separate dev image.** A slimmer dev Dockerfile adds hot-reload and drops the frontend stage (the dev frontend runs as its own node container).

5 — Quality Gates & Local CI Parity

Quality gates run left-to-right — cheapest checks first, locally, long before CI — so most problems are caught before a push and a green local run predicts a green pipeline.

Three layers of gates

1	pre-commit	On every commit: ruff + format, detect-secrets, OpenAPI/typed-client regen, ESLint, branch + large-file guards.
2	check.sh	Pre-push: runs lint → test → build frontend → build image (no push). “Green here means green CI.”
3	CI (ci.yml)	Server-side: ruff, pytest+coverage, ESLint, tsc, Jest, build — the final gate before publish.

Local CI mirror & release scripts

Numbered standalone scripts mirror the CI stages so any failure reproduces locally.

#	Script	Stage
1	01_lint.sh	ruff + ESLint (--max-warnings=0)
2	02_test.sh	ephemeral Postgres + pytest + Jest
3	03_build_frontend.sh	production frontend build
4	04_build_docker.sh	image, tagged from version.json
5	05_push_image.sh	push to the registry (release)

check.sh runs stages 1–4 with no push; release.sh adds the push and refuses to release a dirty working tree, so the published artifact always matches a commit.

Security & contract checks

- **Secret hygiene.** detect-secrets runs against a committed baseline; a guard blocks files over 1000 KB.
- **API contract lockstep.** Any backend change regenerates the OpenAPI schema *and* the frontend's typed client and stages them — CI fails on any drift, so the typed client can never fall behind the server.
- **Static analysis.** Ruff with security and bugbear rules, ESLint at zero warnings, and an optional SonarQube project for deeper QC.

6 — Versioning & Migrations

6.1 — Single-source versioning

`version.json` is the single source of truth and drives three places that must never drift apart: the frontend `package.json`, the backend's `/api/version` endpoint, and the Docker image tag.

```
version.json (1.5.1)
|
+--> frontend package.json
+--> backend /api/version (read at startup)
+--> docker image tag (prod-1.5.1)
```

A helper script bumps the patch version and syncs `package.json`; minor and major bumps are edited directly. The CI build re-reads `version.json` to tag the image, so a release is simply: bump → commit → merge to main → CI publishes the versioned tag.

6.2 — Database migrations (Alembic)

The schema is owned by Alembic (33 revisions), never by an ORM `create_all`. The application runs `alembic upgrade head` on every boot (idempotent), and the migration files are baked into the image so operators can also run upgrades manually against production. Note that the demo-data seeding flag gates seeding only — schema migrations apply on boot regardless of environment.

7 — Observability & Operations

- **Log management.** Every service uses the json-file driver with rotation (max-size 10m, 3–5 files) — bounded disk, no runaway logs.
- **Resource limits.** Per-service memory caps (postgres 512M, backend 1G, frontend 256M).
- **Health checks.** Postgres (`pg_isready`), backend (`/api/health`), frontend (HTTPS probe on `:3443`); Compose gates dependencies on healthy upstreams.
- **TLS.** Nginx terminates TLS 1.2/1.3 with a strong cipher suite and a full security-header set (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy).
- **Operator runbook.** A self-documenting Makefile (`make help`) exposes dev, prod, prod-pull, test, lint, certs, ps, and logs targets.

Recommended next: automated Postgres backups (the prod DB uses a named volume with no backup job yet), centralized metrics and log aggregation, and a CA-signed production certificate.

8 — Implementation Status

This board is the source of truth for what is wired up today versus designed enhancements.

Capability	Status
CI: lint + test + typecheck + build on every PR/push	LIVE
Ephemeral Postgres service container for CI tests	LIVE
Multi-stage prod image with BuildKit + GHA layer cache	LIVE
Image publish to registry on main (versioned tags, retry)	LIVE
Non-root container runtime (appuser, uid 1001)	LIVE
Dev / Prod / Kerberos compose stacks	LIVE
Nginx reverse proxy: TLS, headers, rate-limit, SSE, WS	LIVE
Alembic migrations as schema source of truth (auto on boot)	LIVE
Local CI mirror + release scripts	LIVE
Single-source versioning (version.json)	LIVE
pre-commit: ruff, detect-secrets, contract lockstep, guards	LIVE
Per-service log rotation, memory limits, health checks	LIVE
Dedicated staging environment	PLANNED
Automated database backups	PLANNED
Metrics / centralized logging / alerting	PLANNED
CA-signed production TLS certificate	PLANNED

9 — DevOps Skills Demonstrated

- **CI/CD engineering** — a real GitHub Actions pipeline with service containers, multi-language dependency caching, branch-conditional publish, and a resilient (retry/timeout) registry push.
- **Container craftsmanship** — multi-stage, BuildKit-cached builds; one image serving two services; non-root runtime; cache-friendly layer ordering; CI cache export.
- **Release engineering** — single-source versioning that drives frontend, backend, and image tags; a “no dirty release” guardrail; a reproducible local pipeline that mirrors CI.
- **Reverse-proxy & TLS operations** — Nginx with HTTPS, HSTS/CSP, rate limiting, SSE/WebSocket proxying, immutable static caching, and SPA routing.
- **Schema lifecycle management** — Alembic migrations as the source of truth, applied idempotently on boot and runnable on demand against production.
- **Shift-left quality & security** — layered pre-commit → local-mirror → CI gates, secret scanning with a baseline, and an enforced API-contract lockstep between server and typed client.
- **Operational hygiene** — health-gated dependencies, bounded logging, resource limits, a self-documenting Makefile, and a documented Kerberos-SSO rollout/rollback path.

Area	Technology
CI/CD	GitHub Actions (service containers, caching, conditional publish)
Containers	Docker, BuildKit / Buildx, Docker Compose, GHCR
Runtime	FastAPI / Uvicorn, React, Nginx (TLS), PostgreSQL 15
Migrations	Alembic (auto on boot, baked into image)
Quality gates	pre-commit, Ruff, ESLint, pytest, Jest, tsc, SonarQube
Security	detect-secrets, non-root runtime, loopback-bound backend, Kerberos/AD SSO
Release	version.json single-source, check.sh / release.sh

Product, organization, repository, image, and integration names in this document are anonymized for portfolio use. The architecture, tooling, and workflow reflect a real production system.